

Software Testing

Jibesh Patra, Soumyajit Dey

Week 8 – Software Engineering – CS20202 – Spring 2026 – IIT Kharagpur

Adapted from: Andreas Zeller, Rahul Gopinath, Marcel Böhme, Gordon Fraser, and Christian Holler: "The Fuzzing Book".

Simple Testing

Testing a Function

Does the following code work as expected?

```
1 def my_sqrt(x):  
2     approx = None  
3     guess = x / 2  
4     while approx != guess:  
5         approx = guess  
6         guess = (approx + x / approx) / 2  
7     return approx
```

Run the code with different values of x.

Debug by inserting print statements at different locations of the code.

$$\sqrt{x} * \sqrt{x} = x$$

What does

my_sqrt(2) * my_sqrt(2)

yield?

Automated Test Execution

- We can check using `assert my_sqrt(4) == 2`
- The following helper function:

```
1 def assertEquals(x, y):  
2     assert math.isclose(x, y)
```

```
assertEquals(my_sqrt(2) * my_sqrt(2), 2)
```

- Automatically test for a large number of values

```
1 for n in range(1, 1000):  
2     assertEquals(my_sqrt(n) * my_sqrt(n), n)
```

- We can also integrate the correctness verification in the implementation of the function.

```
1 def my_sqrt(x):  
2     approx = None  
3     guess = x / 2  
4     while approx ≠ guess:  
5         approx = guess  
6         guess = (approx + x / approx) / 2  
7     assertEquals(approx * approx, x)  
8     return approx
```

Limitations of Testing

- Testing can check functionality of a limited number of finite set of input.
- Function fails for untested inputs. Extensive testing may give us a high confidence into the correctness of a program, it does not provide a guarantee that all future executions will be correct.
- Executing `my_sqrt(0)` results in:

```
Traceback (most recent call last):
  File "my_sqrt.py", line 19, in <module>
    my_sqrt(0)
    ~~~~^
  File "my_sqrt.py", line 9, in my_sqrt
    guess = (approx + x / approx) / 2
              ~^~~~~~
ZeroDivisionError: float division by zero
```

Fuzzing



What is Fuzzing?

- Testing programs with random inputs.
- Check if the program **crashes** while running in long sessions.
- The following is a random value generator.

```
1 import random
2
3 def fuzzer(max_length: int = 100, char_start: int = 32, char_range: int = 32)
   → str:
4     string_length = random.randrange(0, max_length + 1)
5     out = ""
6     for i in range(0, string_length):
7         out += chr(random.randrange(char_start, char_start + char_range))
8     return out
```

Example output 448<6 9 759%?? '

Fuzzing External Programs

```
1 import os, subprocess, tempfile
2
3 basename = "temp.txt"
4 tempdir = tempfile.mkdtemp()
5 FILE = os.path.join(tempdir, basename)
6
7 program = "bc"
8 with open(FILE, "w") as f:
9     f.write("2 + 2 + 2\n")
10 result = subprocess.run([program, FILE],
11                          stdin=subprocess.DEVNULL,
12                          stdout=subprocess.PIPE,
13                          stderr=subprocess.PIPE,
14                          universal_newlines=True)
15 print(result.stdout) # prints 6
16 print(result.stderr)
```

- Basic Calculator is a command line utility for performing mathematical calculations.
- We want to test `bc` with randomly generated inputs.

Fuzzing Longer

```
1 trials = 1000
2 program = "bc"
3
4 runs = []
5
6 for i in range(trials):
7     data = fuzzer()
8     with open(FILE, "w") as f:
9         f.write(data)
10    result = subprocess.run([program, FILE],
11                            stdin=subprocess.DEVNULL,
12                            stdout=subprocess.PIPE,
13                            stderr=subprocess.PIPE,
14                            universal_newlines=True)
15    runs.append((data, result))
```

- Run for long and test `bc` with randomly generated inputs.
- Most result in
Parse error: bad....
- `bc` is very well fuzzed.
- Still there is a possibility of finding bugs.



- Transport Layer Security (TLS) protocol implementations uses the popular OpenSSL cryptography library.
- **Heartbleed Vulnerability** in OpenSSL cryptography library.
- Bug due to a missing bounds check in the implementation of the TLS heartbeat extension.
- Undiscovered for two years.

Normal TLS Heartbeat



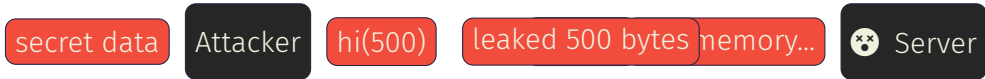
Client sends heartbeat request Server receives and processes Server replies correctly

Normal TLS Heartbeat



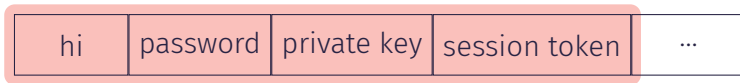
Client sends heartbeat request Server receives and processes Server replies correctly

Heartbleed Attack



Attacker lies about payload length Server copies extra memory Attacker receives sensitive data

Server Heap Memory



Server should return only: **hi**

Missing bounds check → adjacent heap memory leaked.

```
memcpy(response, user_data, user_claimed_length);
```

```
Fix: if (user_claimed_length > actual_data_length) return  
error;
```

Buffer Overflow

- Program writes data to the buffer beyond the allocated memory of the buffer.
- Unexpected behavior in C `char weekday[9]; gets (weekday);` if the input is `"wednesday"`.
- One of the most dangerous Software Weakness.

Buffer Overflow – Example

```
1 #include <stdlib.h>
2 #include <string.h>
3
4 int main(int argc, char** argv) {
5     char *buf = malloc(100);
6     memset(buf, 42, 100);
7
8     int index = atoi(argv[1]);
9     char val = buf[index];
10
11     free(buf);
12     return val;
13 }
```

This C code results in an out-of-bounds error in AddressSanitizer.

Code Coverage

What is Code Coverage?

- How to measure the effectiveness of randomly generated tests?
 - Bug finding ability.
- **Code Coverage** measures parts of a program are actually executed during a test run.
- Acts as a proxy for measuring generated tests. More code coverage can be useful to find bugs.

Code Coverage – Example

 coin_flip.py

```
1 def coin_flip(rand_num):  
2     if rand_num ≥ 50:  
3         return "Heads"  
4     else:  
5         return "Tails"
```

 test_coin_flip.py

```
1 from coin_flip import coin_flip  
2  
3 def test_pass():  
4     assert coin_flip(80) == "Heads"
```

Which part of the program is
executed/covered for

`coin_flip(80)`?

Using pytest to write unit tests.

Code Coverage Report

Terminal Output of:

```
pytest --cov=coin_flip
```

```
==== tests coverage =====  
coverage: platform linux, python 3.13.11-final-0
```

Name	Stmts	Miss	Cover

coin_flip.py	4	1	75%

TOTAL	4	1	75%

```
==== 1 passed in 0.04s =====
```

Can generate HTML report
using

```
--cov-report=html
```

Coverage for **coin_flip.py**: 75%

4 statements 3 run 1 missing 0 excluded

« prev ^ index » next coverage.py v7.13.4, created at

```
1 def coin_flip(rand_num):  
2     if rand_num >= 50:  
3         return "Heads"  
4     else:  
5         return "Tails"
```

« prev ^ index » next coverage.py v7.13.4, created at

How to **improve** the coverage?

-  **Black-Box Testing:** Treat the program as a black box. Do not know the internal structure or code. Only care about **inputs** and **outputs**.
-  **White-Box Testing:** Testing while having access to source code and internal structure of the program.

- Common Gateway Interface (CGI) encoding is used in URLs to encode characters that would be invalid in a URL, such as blanks and certain punctuation.
 - Blanks are replaced by '+'
 - Other invalid characters are replaced by '%xx', where xx is the two-digit hexadecimal equivalent.
- In CGI, text "Hello, world!" is encoded as "Hello%2c+world%21".
- Write a function `cgi_decode` that takes a CGI encoded string and returns the decoded string.

cgi.py

```
1 def cgi_decode(s: str) → str:
2     hex_values = {
3         '0': 0, '1': 1, '2': 2, '3': 3, '4': 4,
4         '5': 5, '6': 6, '7': 7, '8': 8, '9': 9,
5         'a': 10, 'b': 11, 'c': 12, 'd': 13,
6         'e': 14, 'f': 15,
7         'A': 10, 'B': 11, 'C': 12, 'D': 13,
8         'E': 14, 'F': 15,
9     }
10
11     t = ""
12     i = 0
```



```
13 while i < len(s):
14     c = s[i]
15     if c == '+':
16         t += ' '
17     elif c == '%':
18         digit_high, digit_low = s[i + 1], s[i + 2]
19         i += 2
20         if digit_high in hex_values and digit_low in hex_values:
21             v = hex_values[digit_high] * 16 + hex_values[digit_low]
22             t += chr(v)
23         else:
24             raise ValueError("Invalid encoding")
25     else:
26         t += c
27     i += 1
28 return t
```

`cgi_decode("Hello+world")` results in `'Hello world'`.

Derive tests from the **specification**.

1. testing for correct replacement of '+ '.
2. testing for correct replacement of "%xx".
3. testing for non-replacement of other characters.
4. testing for recognition of illegal inputs.



Black-Box Testing – Example

 test CGI.py

```
1 assert cgi_decode('+') == ' '  
2 assert cgi_decode('%20') == ' '  
3 assert cgi_decode('abc') == 'abc'  
4  
5 try:  
6     cgi_decode('%?a')  
7     assert False  
8 except ValueError:  
9     pass
```

Advantages

- Find errors in the *specified* behavior.
- Allows creating tests even before implementation. Test-Driven Development (TDD)

White-Box Testing – Example

Derive tests from the **implementation**.

- *Statement Coverage*: Each statement in the code must be executed by at least one test input.
- *Branch Coverage*: Each branch must be taken by at least one test input.

Which lines in the `cgi_decode()` function do we need to **cover** to achieve this?



White-Box Testing – Example

Coverage for **cgi.py**: 61%

18 statements

11 run

7 missing

0 excluded

« prev ^ index » next coverage.py v7.13.4, created at 2026-02-16 18:00 +0530

```
1 def cgi_decode(s: str) -> str:
2     hex_values = {
3         '0': 0, '1': 1, '2': 2, '3': 3, '4': 4,
4         '5': 5, '6': 6, '7': 7, '8': 8, '9': 9,
5         'a': 10, 'b': 11, 'c': 12, 'd': 13, 'e': 14, 'f': 15,
6         'A': 10, 'B': 11, 'C': 12, 'D': 13, 'E': 14, 'F': 15,
7     }
8
9     t = ""
10    i = 0
11    while i < len(s):
12        c = s[i]
13        if c == '+':
14            t += ' '
15        elif c == '%':
16            digit_high, digit_low = s[i + 1], s[i + 2]
17            i += 2
18            if digit_high in hex_values and digit_low in hex_values:
19                v = hex_values[digit_high] * 16 + hex_values[digit_low]
20                t += chr(v)
21        else:
22            raise ValueError("Invalid encoding")
23    else:
24        t += c
25    i += 1
26    return t
```

- The left figure shows coverage with input `cgi_decode("Hello")`.
- How to improve coverage? Manually write tests.
- Let's Fuzz.

Fuzzing cgi_decode

test CGI.py

```
1 from cgi import cgi_decode
2 from fuzzer import fuzzer
3
4 def test_pass():
5     trial = 2
6     while trial > 0:
7         try:
8             cgi_decode(fuzzer())
9             trial = trial - 1
10        except Exception:
11            pass
```

With *two trials* of fuzzing, we are able to cover 89% and with *four trials* it reaches 100%.

Coverage for **cgi.py**: 89%

18 statements

16 run

2 missing

0 excluded

« prev ^ index » next coverage.py v7.13.4, created at 2026-02-17 09:18 +0530

```
1 def cgi_decode(s: str) -> str:
2     hex_values = {
3         '0': 0, '1': 1, '2': 2, '3': 3, '4': 4,
4         '5': 5, '6': 6, '7': 7, '8': 8, '9': 9,
5         'a': 10, 'b': 11, 'c': 12, 'd': 13, 'e': 14, 'f': 15,
6         'A': 10, 'B': 11, 'C': 12, 'D': 13, 'E': 14, 'F': 15,
7     }
8
9     t = ""
10    i = 0
11    while i < len(s):
12        c = s[i]
13        if c == '+':
14            t += ' '
15        elif c == '%':
16            digit_high, digit_low = s[i + 1], s[i + 2]
17            i += 2
18            if digit_high in hex_values and digit_low in hex_values:
19                v = hex_values[digit_high] * 16 + hex_values[digit_low]
20                t += chr(v)
21        else:
22            raise ValueError("Invalid encoding")
23
24    else:
25        t += c
26        i += 1
27    return t
```

Finding Bugs with Basic Fuzzing

test CGI.py

```
1 from cgi import cgi_decode
2 from fuzzer import fuzzer
3
4 def test_pass():
5     trial = 30
6     while trial > 0:
7         try:
8             cgi_decode(fuzzer())
9             trial = trial - 1
10        except ValueError:
11            pass
```

We explicitly catch the `ValueError` and increase the number of trails.

```
s = ',9 1>;$.75.!>3+7\'\'0:*897!6!?$&/ . ?.! /"(3+%'
```

```
def cgi_decode(s: str) -> str:
    hex_values = {
        '0': 0, '1': 1, '2': 2, '3': 3, '4': 4,
        '5': 5, '6': 6, '7': 7, '8': 8, '9': 9,
        'a': 10, 'b': 11, 'c': 12, 'd': 13, 'e': 14, 'f': 15,
        'A': 10, 'B': 11, 'C': 12, 'D': 13, 'E': 14, 'F': 15,
    }
```

```
    t = ""
    i = 0
    while i < len(s):
        c = s[i]
        if c == '+':
            t += ' '
        elif c == '%':
            digit_high, digit_low = s[i + 1], s[i + 2]
            ~~~~~
```

E **IndexError: string index out of range**

cgi.py:16: IndexError

- Bug in code.
- Not triggered by **Manually Written** tests.

Mutation-Based Fuzzing

Drawbacks of Random Fuzzing

- Most inputs are invalid and as a result are quickly rejected.
- With random inputs, we are not able to explore the input space effectively.
- The **goal** should be to increase chances of obtaining valid inputs.

Mutation-Based Fuzzing

- Start with a set of valid inputs (**seed** inputs).
- Generate new inputs by mutating the seed inputs.
- Test the mutated inputs and collect the ones that are valid.
- Repeat the process.
- As an example, let us fuzz a program that accepts URLs.

URL Parser Example

 urlparse.py

```
1 from urllib.parse import urlparse
2 print(urlparse("https://www.google.com/search?q=fuzzing"))
```

```
ParseResult(scheme='https', netloc='www.google.com', path='/search',
            params='', query='q=fuzzing', fragment='')
```

Parse a URL into 6 components: The result is a named 6-tuple with fields corresponding to the above.

URL Parser Example

 http_program.py

```
1 def http_program(url: str) → bool:
2     supported_schemes = ["http", "https"]
3     result = urlparse(url)
4     if result.scheme not in supported_schemes:
5         raise ValueError("Invalid scheme")
6     if result.netloc == '':
7         raise ValueError("Host must be non-empty")
8     return True
```

The program on left checks if a given URL is valid?

- *Bare minimum*: For a valid URL, we need to start the string with either `http:\\` or `https:\\`.
- If we use the `fuzzer()` function from a past slide, what are the chances of getting a valid URL?

Mutation Testing Schemes

 mutate_inputs.py

```
1 import random
2
3 def delete_random_character(s: str) → str:
4     if s == "":
5         return s
6
7     pos = random.randint(0, len(s) - 1)
8     return s[:pos] + s[pos + 1:]
```

```
23 if __name__ == "__main__":
24     print("---")
25     seed_input = "https://www.google.com/search?q=fuzzing"
26     for i in range(10):
27         x = delete_random_character(seed_input)
28         print(repr(x))
```

Possible schemes:

- Delete Random Character
- Insert Random Character
- Flip Random Character

Mutation Testing Schemes Cont.

 mutate_inputs.py

```
9 def insert_random_character(s: str) → str:
10     pos = random.randint(0, len(s))
11     random_character = chr(random.randrange(32, 127))
12     return s[:pos] + random_character + s[pos:]
13
14 def flip_random_character(s):
15     if s == "":
16         return s
17
18     pos = random.randint(0, len(s) - 1)
19     c = s[pos]
20     bit = 1 << random.randint(0, 6)
21     new_c = chr(ord(c) ^ bit)
22     return s[:pos] + new_c + s[pos + 1:]
```

Adding code for more
mutation schemes.

Randomly Selecting a Scheme

 multiple_mutations.py

```
1 from mutate_inputs import delete_random_character, insert_random_character
   , flip_random_character
2 def mutate(s: str) → str:
3     mutators = [
4         delete_random_character,
5         insert_random_character,
6         flip_random_character
7     ]
8     mutator = random.choice(mutators)
9     return mutator(s)
```

Mutation of URL

```
1 seed_input = "http://www.google.com/search?q=fuzzing"
2 mutations = 50
3 inp = seed_input
4 for i in range(mutations+1):
5     if i % 5 == 0:
6         print(i, "mutations:", repr(inp))
7     inp = mutate(inp)
```

```
0 mutations: 'http://www.google.com/search?q=
fuzzing'
5 mutations: 'http://F/www.gogl%.com/search?qz=
fuzzifg'
...
50 mutations: 'http/0.wwCngo{x/%.#0o\\msm{
KaRch5zqZzifp'}
```

The original input is hardly recognizable but we obtain a variety of inputs.

- The more variety leads to more invalid inputs.
- **Success:** Keep only those inputs that are useful.
- Guide the generation on some pre-defined metrics.
- One option is to use **coverage**.
 - Assume that the specification of the program exists.
 - Can leverage the structure of the program.

Example Fuzzing Tool

american fuzzy lop 0.47b (readpng)		
process timing		overall results
run time : 0 days, 0 hrs, 4 min, 43 sec		cycles done : 0
last new path : 0 days, 0 hrs, 0 min, 26 sec		total paths : 195
last uniq crash : none seen yet		uniq crashes : 0
last uniq hang : 0 days, 0 hrs, 1 min, 51 sec		uniq hangs : 1
cycle progress	map coverage	
now processing : 38 (19.49%)	map density : 1217 (7.43%)	
paths timed out : 0 (0.00%)	count coverage : 2.55 bits/tuple	
stage progress	findings in depth	
now trying : interest 32/8	favored paths : 128 (65.64%)	
stage execs : 0/9990 (0.00%)	new edges on : 85 (43.59%)	
total execs : 654k	total crashes : 0 (0 unique)	
exec speed : 2306/sec	total hangs : 1 (1 unique)	
fuzzing strategy yields	path geometry	
bit flips : 88/14.4k, 6/14.4k, 6/14.4k	levels : 3	
byte flips : 0/1804, 0/1786, 1/1750	pending : 178	
arithmetics : 31/126k, 3/45.6k, 1/17.8k	pend fav : 114	
known ints : 1/15.8k, 4/65.8k, 6/78.2k	imported : 0	
havoc : 34/254k, 0/0	variable : 0	
trim : 2876 B/931 (61.45% gain)	latent : 0	

Figure 1: **American Fuzzy Lop** is an example of a fuzzer that uses coverage (new paths) to guide the generation of inputs.

Tracing Lines While Executing – Example

example.py

```
1 def example_exec(a=2):
2     x = 1
3     y = 2
4     if a == 2:
5         y = 3
6     else:
7         y = 4
8     return x + y
```

trace_lines.py

```
1 import sys
2 from example import example_exec
3 def tracer(frame, event, arg):
4     if event == "line":
5         print(f"Executing line {frame.f_lineno} in {frame.
6             f_code.co_name}")
7     return tracer
8 sys.settrace(tracer)
9 example_exec(2)
10 print("--")
11 example_exec(3)
12 sys.settrace(None)
```

Fuzzing with Coverage (1/3)

fuzz_with_coverage.py

```
1 def fuzz_with_coverage(seed_input, trials=1000):
2     population = [seed_input]
3     coverages_seen = set()
4     all_lines_covered = set()
5     cumulative_coverage = []
6     for i in range(trials):
7         # Pick a previously interesting input at random and mutate it
8         candidate = random.choice(population)
9         for _ in range(random.randint(1, 5)):
10             candidate = mutate(candidate)
11         current_coverage = set()
12         def trace_lines(frame, event, arg):
13             if event == 'line':
14                 current_coverage.add(frame.f_lineno)
15             return trace_lines
16         sys.settrace(trace_lines)
```

Fuzzing with Coverage (2/3)

 fuzz_with_coverage.py

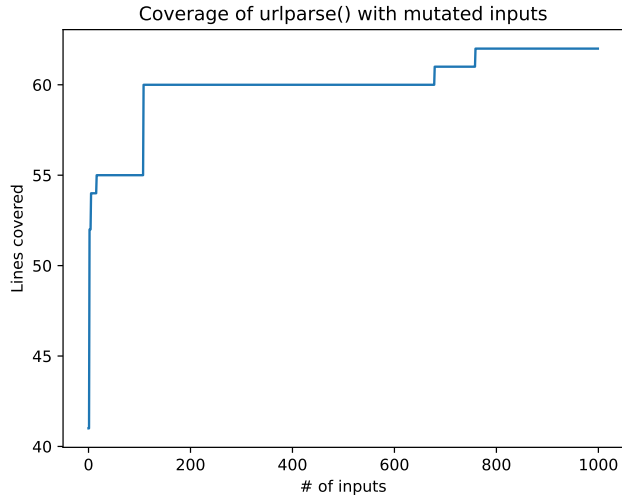
```
17     try:
18         http_program(candidate)
19     except ValueError:
20         pass
21     finally:
22         sys.settrace(None)
23     all_lines_covered.update(current_coverage)
24     cumulative_coverage.append(len(all_lines_covered))
25
26     coverage_frozen = frozenset(current_coverage)
27     if coverage_frozen and coverage_frozen not in coverages_seen:
28         coverages_seen.add(coverage_frozen)
29         population.append(candidate)
30
31     return population, cumulative_coverage
```

Fuzzing with Coverage (3/3)

 fuzz_with_coverage.py

```
32 if __name__ == "__main__":
33     initial_seed = "http://www.google.com/search?q=fuzzing"
34     print("Starting Fuzzing Run ... ")
35
36     successful_inputs, cumulative_coverage = fuzz_with_coverage(
37         initial_seed, trials=1000)
38     print(f"Population grew {len(successful_inputs)} interesting inputs.")
39
40     plt.plot(cumulative_coverage)
41     plt.title('Coverage of urlparse() with mutated inputs')
42     plt.xlabel('# of inputs')
43     plt.ylabel('Lines covered')
44
45     plt.show()
```

Example Output



- Randomly generated inputs are frequently invalid.
- Instead of a pure random generation, we can fuzz *intelligently*.
- Mutations of an existing valid input have much higher chances of being valid.
- We can guide the mutation using coverage.



This is the End

